BECS 22623
Implementation of Numerical Methods using Appropriate software

Department of Mathematics
Faculty of Science
University of Kelaniya

Introduction

The values calculated with

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

are called the **roots** of

$$f(x) = ax^2 + bx + c = 0$$

They represent the values of x such that $f(x) = 0$. For this reason, roots are sometimes called the **zeros** of the equation.

Although the quadratic formula is handy for solving $f(x) = ax^2 + bx + c = 0$, there are many other functions for which the root cannot be determined so easily.

Introduction

One method to obtain an approximate solution is to **plot the function and determine where it crosses the x axis**. This point, which represents the x value for which $f(x) = 0$, is the root.

Graphical Method to Finding Roots

A simple method for obtaining an estimate of the root of the equation $f(x) = 0$ is to make a plot of the function and observe where it crosses the x axis. This point, which represents the x value for which $f(x) = 0$, provides a rough approximation of the root.

Example

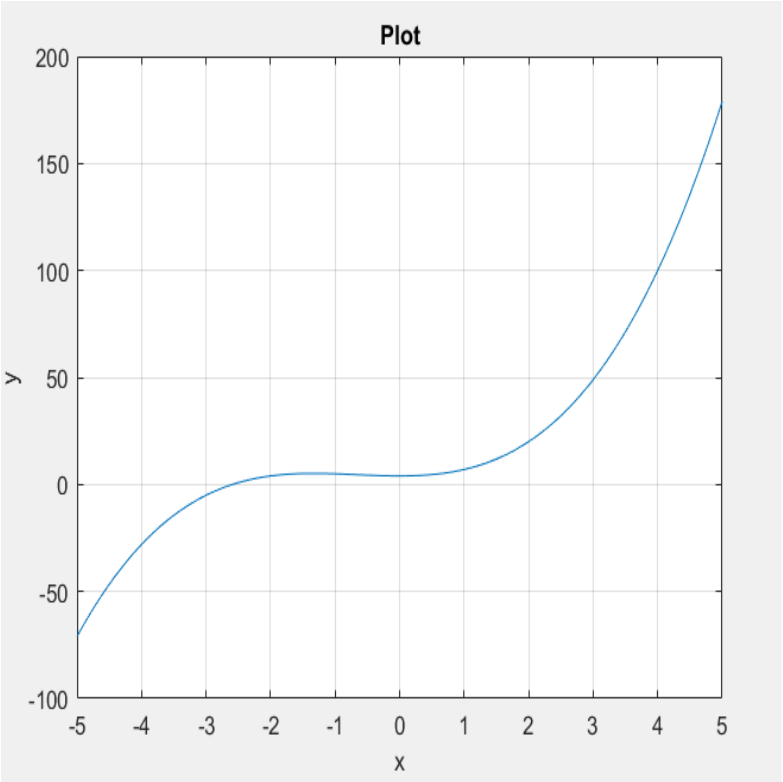
Find the roots of the fourth-order polynomial

$$f(x) = x^3 + 2x^2 + 4 = 0$$

MATLAB Code

```
x=-5:0.1:5;
y = x^3 + 2 * x^2 + 4;
plot(x,y)
xlabel('x')
ylabel('y')
title('Plot')
grid on
```

The resulting plot is shown in the below figure.



Problem

Determine the positive real root of $\ln(x^2) = 0.7$ graphically.

Polynomials

Polynomials

Polynomials are a special type of nonlinear algebraic equation of the general form

$$f_n(x) = a_1 x^n + a_2 x^{n-1} + ... + a_n x + a_{n+1}$$

where n is the order of the polynomial, $\{a_1, a_2, ..., a_n, a_{n+1}\}$ are constant coefficients. In many (but not all) cases, the coefficients will be real. For such cases, the roots can be real and/or complex. In general, an nth order polynomial will have n roots.

Polynomials | MATLAB Function: **roots**

MATLAB has an excellent built-in capability, the **roots** function, for determining all the roots of higher-order polynomials.

Syntax

The roots function has the syntax,

$$x = \text{roots}\ (c)$$

where x is a column vector containing the roots and c is a row vector containing the polynomial's coefficients.

MATLAB code

Example

Find the roots of the third-order polynomial $y = x^3 + 5x + 6$ using function **roots**.

```
c=[1 0 5 6];
x=roots(c)
```

Example

Find the roots of the second-order polynomial $y = x^2 - 4$ using function **roots**.

```
c1=[1 0 -4];
x1=roots(c1)
```

Bracketing Methods and Initial Guess

Bracketing methods. As the name implies, these are based on two initial guesses that "bracket" the root, that is, are on either side of the root.

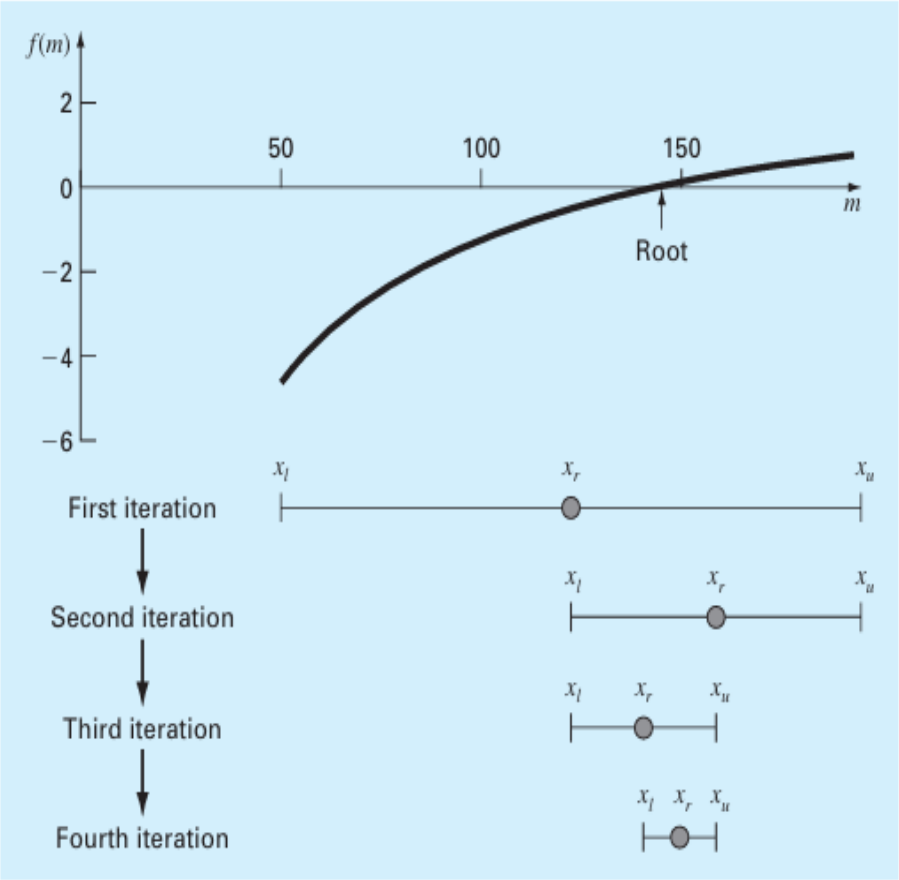
Bracketing Methods and Initial Guess | Bisection Method

The bisection method is a search method in which the interval is always divided in half.

If a function changes sign over an interval, the function value at the midpoint is evaluated. The location of the root is then determined as lying within the subinterval where the sign change occurs.

The subinterval then becomes the interval for the next iteration. The process is repeated until the root is known to the required precision.

Graphical Explanation of the Bisection Method



Explanation of Bisection Method

Example

Find the roots of the second-order polynomial $y = x^2 - 3$ using Bisection method within the interval $[1, 2]$.

Algorithm of the Bisection Method

Bisection Method Algorithm

The steps for applying the bisection method algorithm to find the root of equation f(x)=0 is as follows

Step 1) Choose initial guesses a, b, and tolerance rate e

Step 2) If f(a)f(b) >=0, then the root does not lie in this interval. Thus, there will be no solution.

Step 3) Find the midpoint, c = (a+b)/2

(i) If the function value of the midpoint f(c) = 0, then c is the root. Go to step 5.
(ii) If f(a)f(c) < 0 the root lies between a and c. Then set a = a, b = c.
(iii) Else set a = c, b = b.

Step 4) If the absolute error is higher than the tolerance rate or (b-a) > e, go to step 3.

Step 5) Display c as the approximate root.

MATLAB code of Bisection Method

my_Bisect_new.m × bisection.m ×

```
function root = bisection(f,a,b,n)

if f(a) * f(b)==0
    disp('One of the end point  values is a root. Bisection method will not be applied')
elseif f(a) * f(b)>0
    disp('There is no root in the specified interval. Choose an interval that contains a root')
else
    disp('proceed with the rest of Bisection Method to find the root')
end
% Repeat the Bisection algorithm to find the root

    for i = 1:n
        c = (a+b)/2;
            if f(c) ==0
                fprintf('The root is c = %8.4f',c)
                break
            elseif f(c) * f(a) <0
                b=c;
            else
                a=c;
            end
    end
% Report the final root

root = (a+b)/2;
end
```

Problem

Apply your MATLAB function for bisection method to approximate the root of $p(x) = x^3 - 4x^2 + 5x - 2$ between $x = 1.5$ and $x = 3$ using 10 iterations.

ERRORS

Definition

Numerical errors arise from the use of approximations to represent exact mathematical operations and quantities.

For such errors, the relationship between the exact, or true, result and the approximation can be formulated as

$$\text{True value} = \text{approximation} + \text{error}$$

We find that the numerical error is equal to the discrepancy between the truth and the approximation, as in

$$E_t = \text{True value} - \text{approximation}$$

E_t denotes the exact value of the error.

ERRORS

One way to account for the magnitudes of the quantities being evaluated is to normalize the error to the true value, as in

$$\text{True fractional relative error} = \frac{\text{True value} - \text{approximation}}{\text{True value}}$$

The relative error can also be multiplied by 100% to express it as

$$\epsilon_t = \frac{\text{True value} - \text{approximation}}{\text{True value}} 100\%$$

where ϵ_t denotes the true percent relative error.

ERRORS

An alternative is to normalize the error using the best available estimate of the true value, that is, to the approximation itself, as in

$$\epsilon_a = \frac{\text{approximate error}}{\text{approximation}} 100\%$$

where the subscript a signifies that the error is normalized to an approximate value.

$$\epsilon_a = \frac{\text{present approximation} - \text{previous approximation}}{\text{present approximation}} 100\%$$

ERRORS

Often, when performing computations, we may not be concerned with the sign of the error but are interested in whether the absolute value of the percent relative error is lower than a prespecified tolerance ϵ_s.

The computation is repeated until

$$|\epsilon_a| < \epsilon_s$$

This relationship is referred to as a stopping criterion. If it is satisfied, our result is assumed to be within the prespecified acceptable level ϵ_s.

Open Method

REMARK:
For the bracketing methods, the root is located within an interval prescribed by a lower and an upper bound. Repeated application of these methods always results in closer estimates of the true value of the root. Such methods are said to be convergent because they move closer to the truth as the computation progresses.

In contrast, the open methods require only a single starting value or two starting values that do not necessarily bracket the root. As such, they sometimes diverge or move away from the true root as the computation progresses. However, when the open methods converge they usually do so much more quickly than the bracketing methods.